LangChainPro — Branching, Parallel and Retry Patterns for a Real Customer Support Pipeline

> **Short description:** A customer-support ticket pipeline built only with LangChain LCEL: a structured classifier routes each ticket to its own reply chain, customer lookups run in parallel, transient failures fall back to a safe reply, and every reply streams live and ends in a checked record. No agent loop, no LangGraph.

Tech: Python · LangChain (LCEL) · OpenAI API · Pydantic · **Type:** Single LCEL pipeline (no agent loop, no graph) · **Level:** Intermediate

Overview

LangChainPro processes a queue of support tickets. For each ticket it:

1. **classifies** it — billing, technical or general — with a structured-output chain that also says whether it's **sure**;
2. **routes** it to the reply chain for that category, each with its own prompt;
3. **looks up** the customer's account and recent orders **at the same time** (billing and technical tickets only);
4. **streams** the reply to the screen as it's written;
5. **records** the result as a checked `TicketRecord` — including whether a human should review it, and why.

When the model can't be reached, even after the client's own retries, the ticket gets a safe canned reply instead of a crash, and the record says so.

Why

Most real backend pipelines are this shape: a few fixed categories, a couple of independent lookups, a model call that must survive a bad minute, and one result that a person and a machine both need. None of that needs an agent loop or a graph. LangChainPro shows how far plain LCEL goes — and where it would stop: the moment a step must loop back, pause for a person, or remember state across a restart.

Features

- **Structured classification** — `prompt | model.with_structured_output(TicketCategory)`; only 3 categories can come back, each with a reason and a `sure` flag.
- **Branching by category** — one reply chain and one prompt file per category.
- **Parallel lookups** — `RunnableParallel` runs the account and order lookups at the same time (about 1 s instead of 2 s).
- **Retry, then fall back** — the OpenAI client retries transient errors; after that, `.with_fallbacks()` sends a safe canned reply. Only transient errors fall back — a bad key or a bug still fails loudly.
- **Live streaming** — `.stream()` prints the reply as it's written.
- **One checked record per ticket** — built from the same streamed reply; unclear tickets and fallbacks are flagged for a human, with the reason.

Demo

```
$ python main.py
LangChainPro — 4 tickets in the queue.

Ticket from C-1001: I was charged twice for my Pro plan this month (INV-2201).
Reply: Sorry about the double charge on your Pro plan. I can see two
charges of $29 on 2026-09-01. Our billing team will check them within
2 business days.
  category:      billing
  used fallback: False
  human review:  False

Ticket from C-1004: My invoice and the dashboard numbers don't match. Bug?
Reply: Sorry about the mismatch. Our billing team will check your
invoice within 2 business days.
  category:      billing
  used fallback: False
  human review:  True
    - the ticket could fit more than one category
```

(Log lines are left out and long lines wrapped. Model replies vary.)

Architecture

```
ticket + customer_id
   │
   ▼
classifier chain        prompt | model.with_structured_output()
```

```
   │  TicketCategory        (+ fallback: general, not sure)
   ▼
build_reply_chain(category)
   ├─ billing / technical:
   │     RunnableParallel(ticket, account, orders)
   │       | prompt | model | StrOutputParser
   └─ general:
         prompt | model | StrOutputParser
   │  (+ fallback: the canned reply)
   ▼
.stream()  ── pieces printed live, joined into the reply
   ▼
TicketRecord  (category, reply, used_fallback,
               needs_human_review, review_reasons)
```

- **Retries**: one layer only — the OpenAI client's built-in 2 retries with backoff.
- **Fallbacks**: `.with_fallbacks([...], exceptions_to_handle=TRANSIENT_ERRORS)` — connection errors, timeouts, rate limits and 5xx only.
- **Review rules**: plain code (`review_reasons()`), not a model call.

Project Structure

```
<repo folder>/   (LangChainPro-Branching-...-Customer-Support-Pipeline/)
├── data/
│   └── sample/
│       └── tickets.json       the 4 test tickets
├── scripts/
│   └── time_lookups.py        one after another vs. parallel
├── src/
│   ├── chains/
│   │   ├── __init__.py
│   │   ├── classifier.py      classify_ticket() -> TicketCategory
│   │   ├── fallback.py        with_canned_reply(), with_unsure_category()
│   │   ├── lookups.py         build_lookup_step() (RunnableParallel)
│   │   └── replies.py         build_reply_chain(category)
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py             get_model()
│   ├── prompts/
│   │   ├── billing.txt        the billing reply rules
│   │   ├── classifier.txt     the sorting rules
│   │   ├── general.txt        the general reply rules
│   │   └── technical.txt      the technical reply rules
│   ├── services/
```

```
│   │   ├── __init__.py
│   │   └── pipeline.py        handle_ticket() -> TicketRecord
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py          get_logger()
│   │   └── prompts.py         load_prompt(name)
│   ├── __init__.py
│   ├── config.py              Config, load_config()
│   ├── exceptions.py          MissingConfigError
│   └── schemas.py             TicketCategory, TicketRecord
├── tests/
│   ├── integration/
│   │   └── test_live_pipeline.py real API calls
│   └── unit/
│       ├── test_config.py     no API key needed
│       ├── test_fallback.py   no API key needed
│       ├── test_lookups.py    no API key needed
│       ├── test_review.py     no API key needed
│       └── test_schemas.py    no API key needed
├── .env                       your real key — never committed
├── .env.example               the setting names, no real values
├── .gitignore                 keeps .env, .venv/, __pycache__/ out
├── LICENSE                    lets others reuse your code (MIT)
├── main.py                    runs the ticket queue — the file you run
├── README.md                  the repo's front page
└── requirements.txt           the libraries, for pip install -r
```

Getting Started

Requirements: Python 3.10+ and an OpenAI API key.

```
git clone <this-repo-url>
# the \ joins the two lines into one folder name
cd LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-\
Customer-Support-Pipeline
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # then put your key in .env
```

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

Other settings live in code, each in one place:

Setting	Where	Default
Chat model	`src/models/llm.py` → `MODEL_NAME`	`gpt-4o-mini`
Errors that fall back	`src/chains/fallback.py` → `TRANSIENT_ERRORS`	connection, rate limit, 5xx
The canned reply	`src/chains/fallback.py` → `FALLBACK_REPLY`	"a member of our team will reply…"
Fake lookup delay	`src/chains/lookups.py` → `LOOKUP_DELAY_SECONDS`	`1`
The ticket queue	`main.py` → `SAMPLES_PATH`	`data/sample/tickets.json`

Your own tickets: edit `data/sample/tickets.json` — each ticket needs a `customer_id`, a `ticket` and an `expected` category.

Usage

Run from the repo root:

```
python main.py
```

To see the parallel lookups save time (no API key needed):

```
python -m scripts.time_lookups
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key
python -m tests.unit.test_config
python -m tests.unit.test_schemas
```

```
python -m tests.unit.test_lookups
python -m tests.unit.test_fallback
python -m tests.unit.test_review

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_pipeline
```

`test_fallback.py` forces a real outage (a model pointed at a local port where nothing listens) and checks the fallback fires — and that a working chain and a real bug don't trigger it.

Error Handling

Situation	What happens
`OPENAI_API_KEY` missing	Exits at startup with one clear line
Invalid API key	Caught in the loop; exits cleanly — never hidden by a fallback
Timeout, connection error, rate limit, 5xx	The client retries 2 times; then the fallback runs
The classifier can't be reached	The ticket becomes "general, not sure" and still gets a reply
A reply chain can't be reached	The canned reply is sent; the record has `used_fallback=True`
The classifier's answer doesn't fit the schema	Caught as `ValidationError`; "general, not sure"
A bug in the code (e.g. `KeyError`)	Raised as normal — fallbacks never hide it
Unknown customer id	The lookups say "No account found" instead of failing

Design Decisions

- **LCEL, not LangGraph** — the pipeline never loops back, never pauses for a person, and keeps no state between runs; a graph would add cost and no benefit.
- **Branching with a prompt file per category** — the category name is the prompt name; a plain `if` decides only whether lookups run.
- **One retry layer** — the client already retries; `.with_retry()` on top would multiply calls.
- **Fall back only on transient errors** — a canned reply must never hide a bad key or a bug.
- **A canned reply that promises nothing** — safe even for a double charge, and always flagged for a human.
- **The record comes from the streamed reply** — not a second model call that could disagree with it.

- **Review rules in plain code** — each rule is testable and names itself in `review_reasons`.

Limitations and Roadmap

- The account and order lookups are stand-ins with fixed data and a fake delay.
- Tickets come from a JSON file; nothing is sent to a real customer or saved to a real ticketing system.
- Tickets are processed one at a time.
- Possible next steps: real lookups against a database or API, saving records to a ticketing system, `.batch()` for a large queue, a web UI for the review queue.

License

MIT — see `LICENSE`.